Circles Rendering

Parallel Rendering of Random Circles with Alpha-Blending

Nome Cognome
Dipartimento di Informatica — Università degli Studi di XXXX
nome.cognome@studenti.unixxx.it

Abstract*—*Questo lavoro affronta il rendering di N cerchi casuali con alpha-blending su un'immagine* 1920×1080*. Partendo da un'implementazione sequenziale di riferimento (AoS), vengono introdotte cinque ottimizzazioni progressive: sostituzione della struttura dati in Structure of Arrays (SoA), parallelizzazione tramite OpenMP con granularità a livello di cella di griglia e a livello di pixel, e vettorizzazione SIMD del test geometrico. Il benchmark, condotto variando il numero di cerchi (1k–50k), la GridSize (4–32) e il numero di thread OpenMP (4–16), mostra speedup fino a* ***5.4$\times$ *rispetto alla baseline sequenziale. La combinazione SoA + loop 1D su pixel + SIMD risulta la variante più efficiente in quasi tutte le configurazioni testate.*

I. Introduzione

Il rendering raster di primitive geometriche con alpha-blending è un problema classico in computer graphics che si presta naturalmente alla parallelizzazione: ogni pixel può essere calcolato indipendentemente dagli altri. In questo progetto si affronta il caso specifico di N cerchi, ciascuno con posizione (x, y, z), raggio e colore RGBA, da comporre su un'immagine 1920×1080 tramite il modello Porter-Duff *over*.

L'obiettivo del lavoro è duplice: (i) implementare e confrontare sei varianti di renderer con livelli crescenti di ottimizzazione; (ii) quantificare e motivare il guadagno prestazionale di ciascuna al variare dei parametri del problema.

Il documento è strutturato come segue: la Sezione II descrive l'architettura comune e le strutture dati; la Sezione III illustra le sei varianti; la Sezione IV riporta e analizza il benchmark; la Sezione V discute i risultati; la Sezione VI conclude.

II. Architettura del Sistema

A. Struttura dati: AoS vs. SoA

La rappresentazione in memoria dei cerchi è la prima scelta di progetto. Nella variante *Array of Structures* (AoS) ogni cerchio occupa un blocco contiguo di 12 byte (`short x,y,z,radius` più `uchar r,g,b,a`). Nella variante *Structure of Arrays* (SoA) ogni attributo è un array separato: `short* x, *y, *z, *radius; uchar* r,*g,*b,*a`.

Il vantaggio della SoA emerge nel loop più interno: il test geometrico $\Delta y^2 + \Delta x^2 \leq r^2$ accede solo a `x[]`, `y[]` e `radius[]`, caricando linee di cache omogenee e dense. Con AoS ogni cache line porta in memoria anche i canali colore, inutili in questa fase. La SoA è inoltre il layout ottimale per la vettorizzazione SIMD, poiché gli operandi sono contigui e dello stesso tipo.

B. Accelerazione spaziale: griglia uniforme

Il costo naive del rendering è $O(N \cdot W \cdot H)$. Per ridurlo, l'immagine viene suddivisa in una griglia uniforme di $G \times G$ celle (`GRID_SIZE`$= G$). Ogni cerchio viene pre-assegnato alle sole celle intersecate dalla sua bounding box; al rendering ogni pixel esamina solo i cerchi della propria cella.

I cerchi di ciascuna cella vengono ordinati per z decrescente tramite quicksort, così il blending procede automaticamente dal retro verso il fronte. Un cerchio che tocca più celle è replicato; il rapporto di ridondanza è contenuto perché il raggio massimo è fissato a $\min(W, H)/(2G)$.

Il parametro G è il principale trade-off: valori alti riducono i cerchi per cella ma contraggono le celle, limitando il parallelismo e aumentando l'overhead di costruzione. Come mostrato in Sezione IV, $G = 8$ rappresenta il punto di ottimo per i carichi considerati.

C. Alpha-blending

Il modello Porter-Duff *over* combina colore sorgente e destinazione:

$$\alpha_{\text{out}} = \alpha_s + \alpha_d (1 - \alpha_s)$$

$$C_{\text{out}} = \frac{C_s\, \alpha_s + C_d\, \alpha_d (1 - \alpha_s)}{\alpha_{\text{out}}}$$

La dipendenza seriale su `finalColor` impedisce la vettorizzazione del blending, che rimane sequenziale per pixel. Il test geometrico, privo di dipendenze tra cerchi, è invece completamente vettorizzabile.

III. Varianti Implementate

Le sei varianti condividono la stessa gerarchia di classi: `Renderer` (base astratta) $\rightarrow$ renderer specializzati. La Tabella 1 ne riassume le caratteristiche principali.

Table 1: Riepilogo delle varianti implementate.

Variante	Layout	Parallelismo	SIMD
Sequential	AoS	—	No
SeqMemOpt	SoA	—	No
Parallel	AoS	OMP celle	No
ParOverPixels	AoS	OMP pixel	No
ParallelSoA	SoA	OMP celle	Sì
ParSoAOverPixels	SoA	OMP pixel	Sì

A. Sequential Base

Implementazione di riferimento con struttura AoS e nessun parallelismo. Itera su celle, poi su pixel della cella, poi su cerchi con test geometrico e blending. Serve da baseline per il calcolo di tutti gli speedup.

B. Sequential MemOpt (SoA)

Stessa logica della baseline ma con struttura SoA. L'accesso nel loop sui cerchi diventa sequenziale su array distinti, favorendo il prefetching hardware e la vettorizzazione automatica del compilatore.

C. Parallel Base (OpenMP su celle)

Applica `#pragma omp parallel for collapse(2) schedule(dynamic)` ai due loop più esterni (riga e colonna di griglia). Ogni thread gestisce in autonomia un sottoinsieme di celle; l'assenza di dipendenze tra celle garantisce la correttezza senza sincronizzazioni esplicite. Il limite principale è che il numero di task paralleli è G^2: con $G = 4$ e 16 thread si hanno solo 16 task, saturando rapidamente il parallelismo disponibile.

D. Parallel Over Pixels (OpenMP su pixel)

Per aumentare la granularità, il quadruplo loop annidato (riga, colonna, i, j) viene fuso in un unico loop lineare su tutti i $W \times H$ pixel. Ogni thread calcola autonomamente la cella di appartenenza dalla posizione del pixel tramite divisione intera. Questo produce circa 2M iterazioni indipendenti, permettendo un bilanciamento dinamico molto più fine.

E. Parallel SoA (OpenMP + SoA + SIMD)

Combina il parallelismo su celle con la struttura SoA e la direttiva `#pragma omp simd` sul loop dei cerchi (Fase 1: test geometrico). Il loop è privo di dipendenze tra le iterazioni su k ed è espresso su array contigui omogenei, condizioni ideali per il compilatore. Il vettore temporaneo `hits[]` è allocato una volta per thread fuori dal loop parallelo (`std::vector` privato), eliminando allocazioni dinamiche nel percorso critico. La Fase 2 (blending) rimane sequenziale.

F. Parallel SoA Over Pixels

Variante più completa: loop 1D su pixel per massimizzare la granularità OpenMP, struttura SoA per la cache locality, e `omp simd` per la fase geometrica. Il vettore `hits[]` è dichiarato nella sezione `parallel` come variabile privata per thread (schema split `parallel/for`), garantendo assenza di false condivisioni.

IV. Risultati Sperimentali

A. Setup

I test sono stati eseguiti variando: $N \in \{1\text{k}, 5\text{k}, 10\text{k}, 20\text{k}, 50\text{k}\}$ cerchi; $G \in \{4, 8, 16, 32\}$; thread OpenMP $T \in \{4, 8, 12, 16\}$. Ogni configurazione è misurata una volta cronometrando solo `generateImage()` con `chrono::high_resolution_clock`. Lo speedup è definito come $S = T_{\text{seq}}/T_{\text{var}}$ con la stessa G.

B. AoS vs. SoA nel sequenziale

La Figura 1 confronta Sequential e SeqMemOpt a $G = 8$. Il guadagno SoA è modesto a carico basso ($\approx$4% con $N = 1$k) ma cresce dove il loop interno è lungo: con $N = 50$k e $G = 4$ la riduzione è del 34% (407 $\rightarrow$ 268 s). A $G = 8$ il vantaggio scende a $\approx$10% perché le celle più piccole accorciano il loop sui cerchi. Questo conferma che la SoA porta benefici concreti solo quando la bandwidth di memoria diventa il collo di bottiglia, ovvero con molti cerchi per cella.

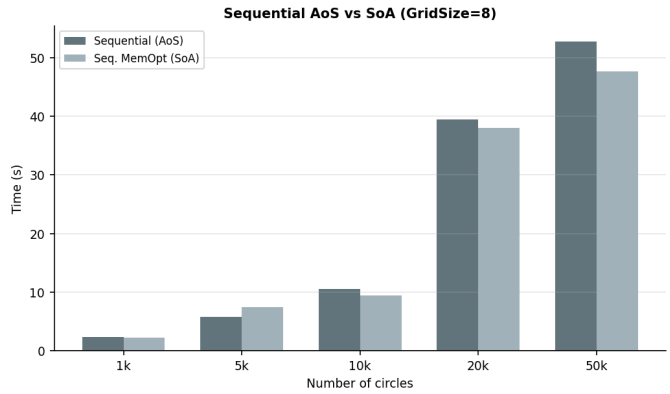

Figure 1: Confronto Sequential AoS vs. SeqMemOpt SoA al variare di N. $G = 8$. Il guadagno SoA cresce con il carico.

C. Effetto della GridSize

La GridSize è il parametro con il maggiore impatto sulle prestazioni assolute. La Tabella 2 riporta i tempi per $N = 10$k cerchi con 8 thread al variare di G.

Table 2: Tempo (s) al variare di G. $N = 10$k, 8 thread.

G	Seq	SMem	Par	OvPx	SoA	SoA+Px
4	37.39	33.80	10.36	9.80	9.71	9.39
8	10.51	9.42	3.08	2.91	2.74	2.74
16	3.03	2.69	0.86	0.85	0.77	0.81
32	0.97	0.85	0.25	0.31	0.24	0.31

La riduzione del tempo è quasi quadratica con G: raddoppiando G da 4 a 8 il tempo sequenziale cala di un fattore $\approx$3.5$\times$, poiché il loop interno si accorcia proporzionalmente al numero di cerchi per cella. Oltre $G = 16$ il beneficio si attenua: la costruzione della griglia e la dimensione ridotta delle celle iniziano a controbilanciare il guadagno. La Figura 2 conferma che questo andamento è coerente su tutte le varianti parallele, con $G = 8$ come punto di ottimo.

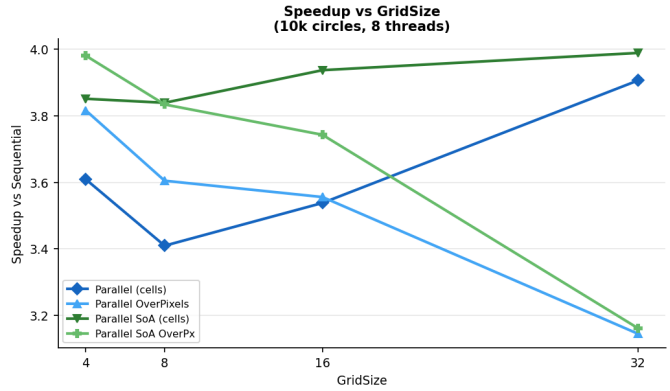

Figure 2: Speedup delle varianti parallele rispetto a Sequential al variare di G. $N = 10$k, 8 thread. Tutte le varianti raggiungono il picco a $G = 8$; con $G = 32$ l'overhead di griglia erode parte del guadagno.

D. Confronto generale tra le varianti

La Figura 3 mostra il tempo in scala logaritmica per tutte le varianti a $G = 8$ e 16 thread. L'ordine delle prestazioni è stabile al crescere di N:

$$\text{SoA+Px} \approx \text{SoA} < \text{OvPx} < \text{Par} < \text{SMem} < \text{Seq}$$

Il gap tra le varianti SoA e le corrispondenti AoS è del 5–10% e si accentua a carichi pesanti, dove la bandwidth di memoria limita le versioni AoS. Il distacco tra versioni parallele e sequenziali è invece netto e cresce con N.

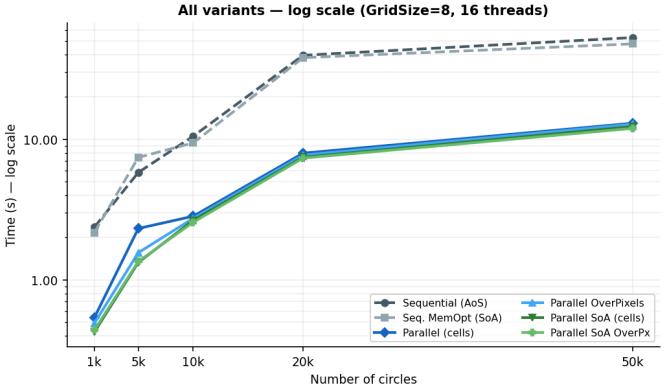

Figure 3: Tempo di rendering (scala log) per tutte le varianti. $G = 8$, 16 thread. La scala logaritmica evidenzia il distacco crescente tra versioni parallele e sequenziali.

E. Speedup al variare dei thread

Lo speedup è riportato in Figura 4 per $N = 20$k e $G = 8$, e numericamente in Tabella 3 per le configurazioni più significative. Lo scaling è sub-lineare in accordo con la legge di Amdahl: passando da 4 a 16 thread lo speedup aumenta di soli $\approx 1.5\times$, ben lontano dal limite teorico ($4\times$). La frazione seriale non parallelizzabile — blending e costruzione della griglia — vincola lo speedup massimo osservato a ≈ 5–$6\times$.

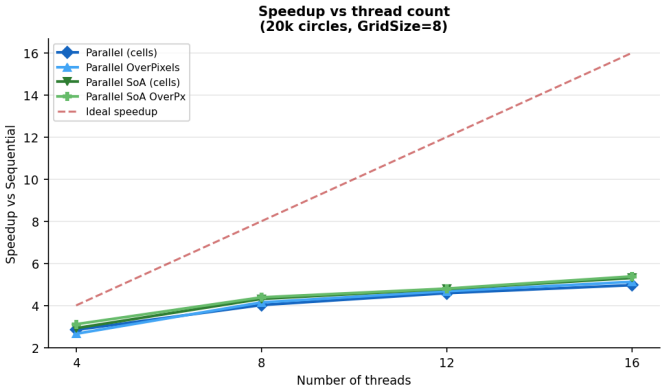

Figure 4: Speedup al variare di T. $N = 20$k, $G = 8$. Il tratteggio rosso indica lo speedup ideale lineare. Le varianti SoA sono sistematicamente superiori.

Table 3: Speedup vs. Sequential. $G = 8$. In grassetto: valori $> 4\times$.

N	T	Par	OvPx	SoA	SoA+Px
1k	4	3.21	3.22	3.61	2.06
	8	3.27	4.00	4.52	3.62
	12	4.06	4.60	**5.30**	4.60
	16	4.39	4.81	**5.56**	**5.39**
10k	4	3.08	3.08	3.13	3.08
	8	3.41	3.61	3.84	3.83
	12	3.75	3.92	**4.04**	3.97
	16	3.71	3.84	3.97	**4.09**
20k	4	2.84	2.65	2.92	3.10
	8	**4.01**	**4.14**	**4.31**	**4.38**
	12	4.58	**4.67**	**4.78**	**4.80**
	16	**4.96**	**5.12**	**5.30**	**5.38**
50k	4	3.14	3.11	3.16	3.33
	8	3.92	3.94	**4.10**	**4.21**
	12	3.97	**4.09**	**4.18**	**4.36**
	16	**4.06**	**4.12**	**4.29**	**4.42**

I valori in grassetto (speedup $> 4\times$) mostrano che la soglia viene raggiunta stabilmente solo con $N \geq 10$k e $T \geq 8$, e che le varianti SoA la superano con thread minori rispetto alle AoS. Con $N = 1$k e pochi thread lo speedup è basso perché il lavoro per pixel è troppo esiguo per ammortizzare l'overhead OpenMP.

F. Efficienza parallela

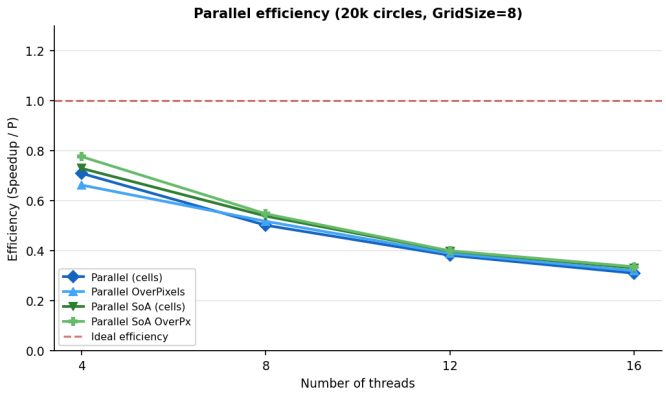

Figure 5: Efficienza $E = S/T$ al variare di T. $N = 20$k, $G = 8$. L'efficienza ideale ($E = 1$) è in tratteggio rosso.

La Figura 5 mostra l'efficienza $E = S/T$. Tutte le varianti degradano da ≈ 0.7–0.8 con 4 thread a ≈ 0.3–0.4 con 16 thread. La degradazione è attesa: all'aumentare di T il costo di sincronizzazione e scheduling cresce mentre la frazione seriale rimane fissa. Le varianti SoA mantengono un'efficienza costantemente superiore di 3–5 punti percentuali, a conferma che la migliore cache locality riduce i tempi di attesa in memoria e rende il lavoro per thread più denso.

G. Heatmap delle prestazioni di Parallel SoA

La Figura 6 mappa il tempo di Parallel SoA nello spazio (N, G) a 8 thread. Ogni cella riporta il tempo in secondi; il gradiente di colore è in scala logaritmica.

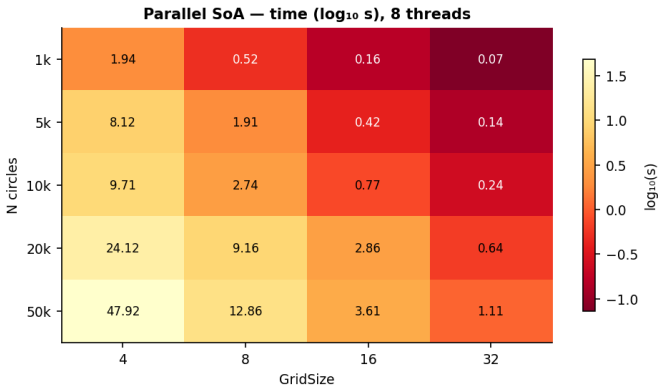

Figure 6: Heatmap di Parallel SoA: tempo ($\log_{10}$ s) in funzione di N (righe) e G (colonne). 8 thread.

Si legge chiaramente che G domina rispetto a N: aumentare G da 4 a 8 dimezza il tempo più efficacemente di qualsiasi altra ottimizzazione su qualsiasi carico. Il peggioramento visibile con $G = 32$ e N piccolo rispecchia l'overhead di costruzione della griglia quando il numero medio di cerchi per cella scende sotto l'unità: la struttura dati è troppo grande rispetto al lavoro utile che genera.

V. Discussione

1) Legge di Amdahl— La saturazione dello speedup intorno a 5–6× con 16 thread suggerisce una frazione seriale $f \approx 17$–20%, attribuibile principalmente al blending (dipendenza su `finalColor`) e alla costruzione sequenziale della griglia.

2) Granularità e scheduling— Il loop 1D su pixel genera ≈2M task, permettendo a `schedule(dynamic)` di bilanciare carichi eterogenei (cerchi concentrati in zone dell'immagine). Il loop su celle produce solo G^2 task: troppo pochi con G piccolo, ma sufficienti con $G = 32$ (1024 celle), dove le due strategie si equivalgono e la versione su celle può risultare più veloce per il minore overhead di scheduling.

3) SIMD e layout di memoria— La direttiva `omp simd` è efficace solo con SoA: gli array `x[]`, `y[]`, `radius[]` sono contigui e omogenei, permettendo di caricare 4–8 `short` per istruzione SIMD. Con AoS i dati sarebbero interlacciati e il compilatore non potrebbe vettorizzare.

4) Anomalie con $G = 4$— Alcune configurazioni con $G = 4$ e $N = 5$k mostrano tempi paralleli superiori al sequenziale. Questo è attribuibile a contesa di cache tra thread: a $G = 4$ ogni cella copre 480×270 pixel, troppo grande per stare in cache L1/L2, e i thread si contendono la cache L3.

VI. Conclusioni

Il progetto ha confrontato sei varianti di un renderer di cerchi con alpha-blending, con i seguenti risultati principali:

- La **griglia spaziale** è l'ottimizzazione più impattante: $G = 8$ riduce il tempo di un fattore 3–4× rispetto a $G = 4$.
- La **struttura SoA** abilita la vettorizzazione SIMD e migliora la cache locality, con guadagni del 5–34% rispetto all'AoS.
- Il **parallelismo OpenMP** scala sub-linearmente (Amdahl): il blending sequenziale limita lo speedup massimo a ≈5–6×.
- Il **loop 1D su pixel** migliora il bilanciamento del carico rispetto al loop su celle, soprattutto con G piccolo.
- La variante **Parallel SoA Over Pixels** è la più efficiente in quasi tutte le configurazioni testate.